Batch: B1 Roll No.: 16010122158

Experiment No. 5

TITLE : To perform forecasting using time series analysis

AIM: To perform forecasting using time series analysis

Expected OUTCOME of Experiment:
CO4: Perform Time series Analytics and forecasting

Books/ Journals/ Websites referred:

Students have to list.

Pre Lab/ Prior Concepts:

Students should have a basic understanding of: Time series Analytics and forecasting

Procedure:

Data set Used: Air Quality

Step1: Select and Load the dataset

(Students should write the code and output)

```
[1]  import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
     from statsmodels.tsa.arima.model import ARIMA
     from statsmodels.tsa.seasonal import seasonal_decompose
```

```
[9]  # Load the data
     import pandas as pd

     # Load the dataset
     data = pd.read_csv('AirQualityUCIfinal.csv')
```

```
# Check the columns to ensure 'Date' and 'Time' exist
print(data.columns)

# Combine 'Date' and 'Time' into a single 'datetime' column if they exist
if 'Date' in data.columns and 'Time' in data.columns:
    data['datetime'] = pd.to_datetime(data['Date'] + ' ' + data['Time'], format='%d-%m-%Y %H:%M:%S')

    # Set the 'datetime' column as the index
    data.set_index('datetime', inplace=True)

    # Drop the original 'Date' and 'Time' columns as they are no longer needed
    data.drop(columns=['Date', 'Time'], inplace=True)

# Optional: Convert columns to numeric if they are not already
data = data.apply(pd.to_numeric, errors='coerce')

# Display the first few rows of the dataset
print(data.head())

series = data['NO2(GT)']  # Adjust column names as needed
```

```
Index(['CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)',
       'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)',
       'T', 'RH', 'AH'],
      dtype='object')
                     CO(GT)  PT08.S1(CO)  NMHC(GT)  C6H6(GT)  PT08.S2(NMHC)  \
datetime
2004-03-10 18:00:00     2.6         1360       150      11.9           1046
2004-03-10 19:00:00     2.0         1292       112       9.4            955
2004-03-10 20:00:00     2.2         1402        88       9.0            939
2004-03-10 21:00:00     2.2         1376        80       9.2            948
2004-03-10 22:00:00     1.6         1272        51       6.5            836

                     NOx(GT)  PT08.S3(NOx)  NO2(GT)  PT08.S4(NO2)  \
datetime
2004-03-10 18:00:00      166          1056      113          1692
2004-03-10 19:00:00      103          1174       92          1559
2004-03-10 20:00:00      131          1140      114          1555
2004-03-10 21:00:00      172          1092      122          1584
2004-03-10 22:00:00      131          1205      116          1490

                     PT08.S5(O3)     T    RH      AH
datetime
2004-03-10 18:00:00         1268  13.6  48.9  0.7578
2004-03-10 19:00:00          972  13.3  47.7  0.7255
2004-03-10 20:00:00         1074  11.9  54.0  0.7502
2004-03-10 21:00:00         1203  11.0  60.0  0.7867
2004-03-10 22:00:00         1110  11.2  59.6  0.7888
```

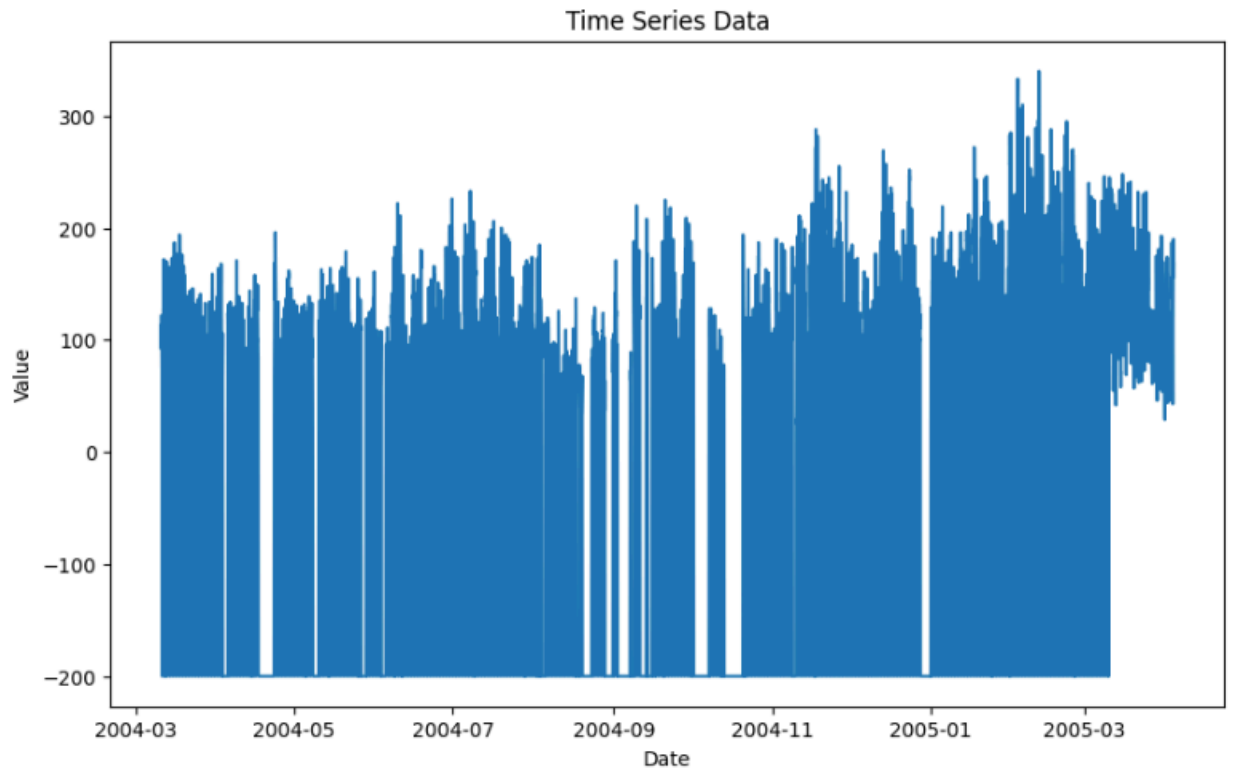
Step2: Visualize the data

(Students should write the code and output)

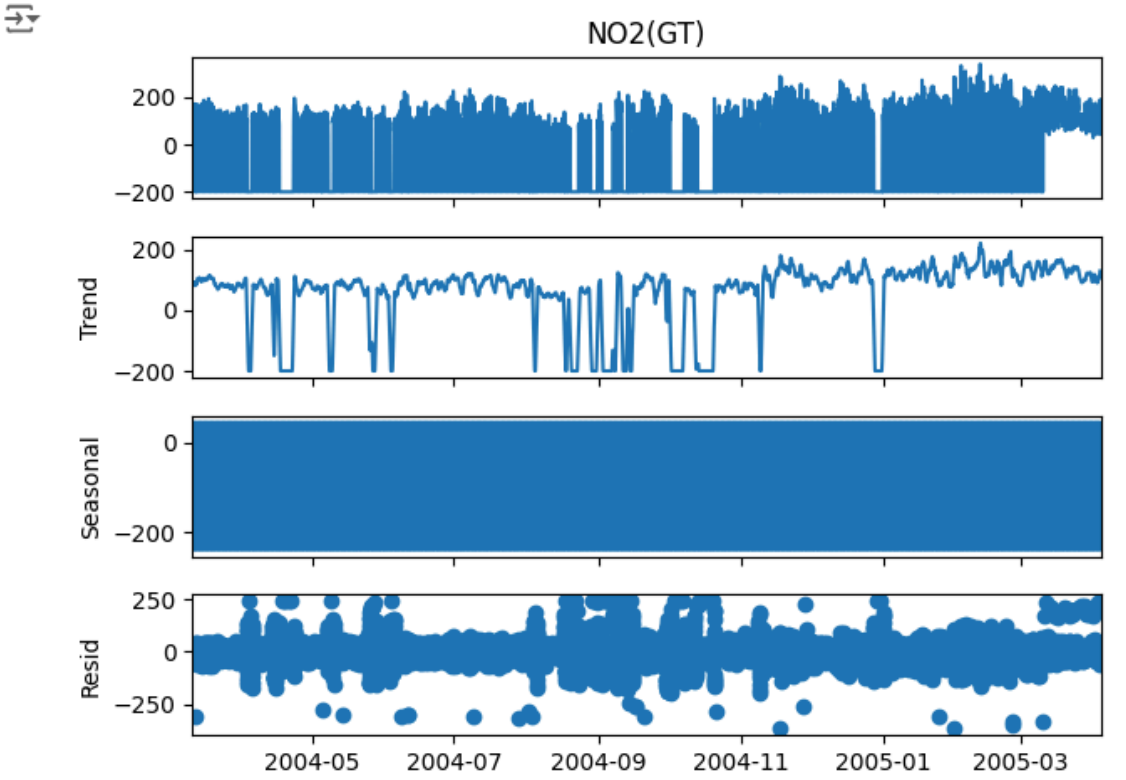
```
[15] print(data.columns)
```

```
Index(['CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)',
       'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)',
       'T', 'RH', 'AH'],
      dtype='object')
```

```
# Visualize the data
plt.figure(figsize=(10, 6))
plt.plot(series)
plt.title('Time Series Data')
plt.xlabel('Date')
plt.ylabel('Value')
plt.show()
```



```
[18] # Decompose the data
     decomposition = seasonal_decompose(series, model='additive')
     fig = decomposition.plot()
     plt.show()
```



Step 3: Fit the model (ARIMA Model is Used)

(Students should write the code and output)

```
[19] # Fit ARIMA model
     model = ARIMA(series, order=(3, 1, 0))  # Adjust the order as needed
     model_fit = model.fit()
```

```
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency h will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency h will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency h will be used.
  self._init_dates(dates, freq)
```

```
[20] # Summary of the model
     print(model_fit.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                  NO2(GT)   No. Observations:                 9357
Model:                 ARIMA(3, 1, 0)   Log Likelihood              -52949.067
Date:                Thu, 03 Oct 2024   AIC                         105906.134
Time:                        11:25:34   BIC                         105934.710
Sample:                    03-10-2004   HQIC                        105915.839
                         - 04-04-2005
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.4744      0.006    -78.679      0.000      -0.486      -0.463
ar.L2         -0.2122      0.011    -20.090      0.000      -0.233      -0.192
ar.L3         -0.0826      0.013     -6.414      0.000      -0.108      -0.057
sigma2      4822.9487     28.299    170.428      0.000    4767.484    4878.414
===================================================================================
Ljung-Box (L1) (Q):                   0.12   Jarque-Bera (JB):             49931.76
Prob(Q):                              0.73   Prob(JB):                         0.00
Heteroskedasticity (H):               1.11   Skew:                            -2.17
Prob(H) (two-sided):                  0.00   Kurtosis:                        13.45
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Step4: Forecast future values

(Students should write the code and output)

```
[21] # Forecast future values
     forecast_steps = 3  # Number of periods to forecast
     forecast, stderr, conf_int = model_fit.forecast(steps=forecast_steps)
```

Step 5: Create a DataFrame for the forecast

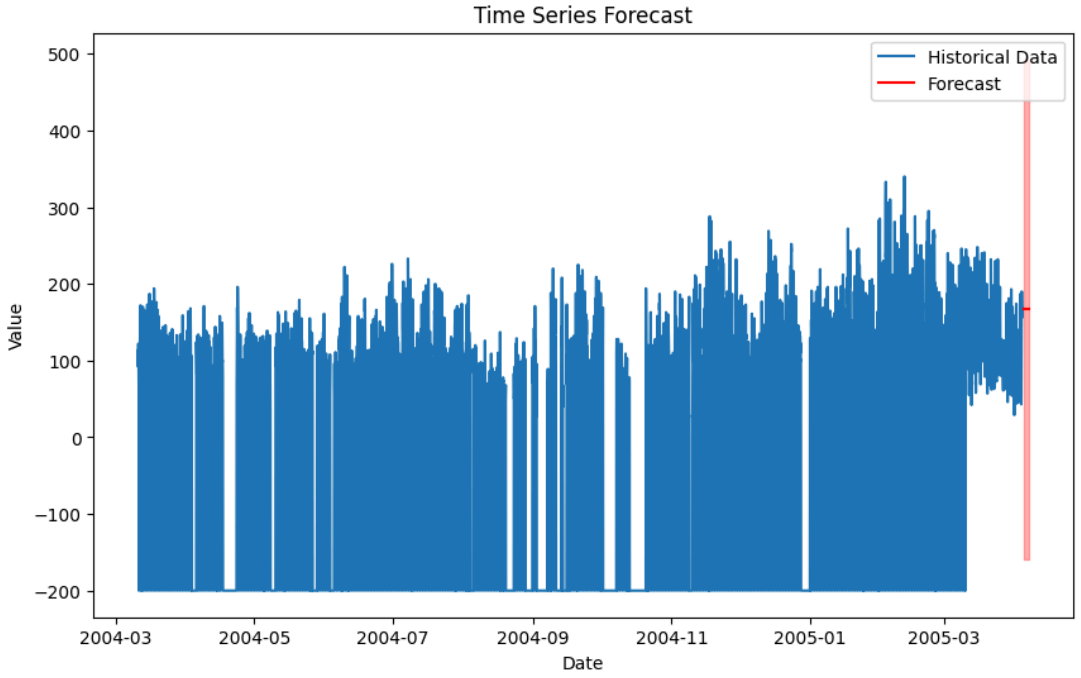
(Students should write the code and output)

```
[22] # Create a DataFrame for the forecast
     forecast_index = pd.date_range(start=series.index[-1] + pd.Timedelta(days=1), periods=forecast_steps, freq='D')
     forecast_series = pd.Series(forecast, index=forecast_index)
```

Step 6: Plot the results

(Students should write the code and output)

```
[23] # Plot the results
     plt.figure(figsize=(10, 6))
     plt.plot(series, label='Historical Data')
     plt.plot(forecast_series, color='red', label='Forecast')
     plt.fill_between(forecast_series.index, forecast_series - 1.96 * stderr, forecast_series + 1.96 * stderr, color='red', alpha=0.3)
     plt.title('Time Series Forecast')
     plt.xlabel('Date')
     plt.ylabel('Value')
     plt.legend()
     plt.show()
```



Students have to perform all the tasks illustrated above by choosing any other time series related dataset.

Implementation details:
Step 1)

```
# To install the library
!pip install pmdarima

--NORMAL--

Requirement already satisfied: pmdarima in /usr/local/lib/python3.10/dist-packages (2.0.4)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.4.2)
Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (3.0.11)
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.26.4)
Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.2)
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.5.2)
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.13.1)
Requirement already satisfied: statsmodels>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (0.14.3)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.3)
Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (71.0.4)
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (24.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.22->pmdarima) (3.5.0)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-packages (from statsmodels>=0.13.2->pmdarima) (0.5.6)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from patsy>=0.5.6->statsmodels>=0.13.2->pmdarima) (1.16.0)
```

```
[6]
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.seasonal import seasonal_decompose

# Load the data
import pandas as pd

# Load the dataset
data = pd.read_csv('AirQualityUCIfinal.csv')

# Combine 'Date' and 'Time' into a single 'datetime' column
data['datetime'] = pd.to_datetime(data['Date'] + ' ' + data['Time'], format='%d-%m-%Y %H:%M:%S')

# Set the 'datetime' column as the index
data.set_index('datetime', inplace=True)

# Drop the original 'Date' and 'Time' columns as they are no longer needed
data.drop(columns=['Date', 'Time'], inplace=True)

# Optional: Convert columns to numeric if they are not already
data = data.apply(pd.to_numeric, errors='coerce')

# Display the first few rows of the dataset
print(data.head())
```

```
--VISUAL--
```

datetime	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	PT08.S2(NMHC)	\
2004-03-10 18:00:00	2.6	1360	150	11.9	1046	
2004-03-10 19:00:00	2.0	1292	112	9.4	955	
2004-03-10 20:00:00	2.2	1402	88	9.0	939	
2004-03-10 21:00:00	2.2	1376	80	9.2	948	
2004-03-10 22:00:00	1.6	1272	51	6.5	836	

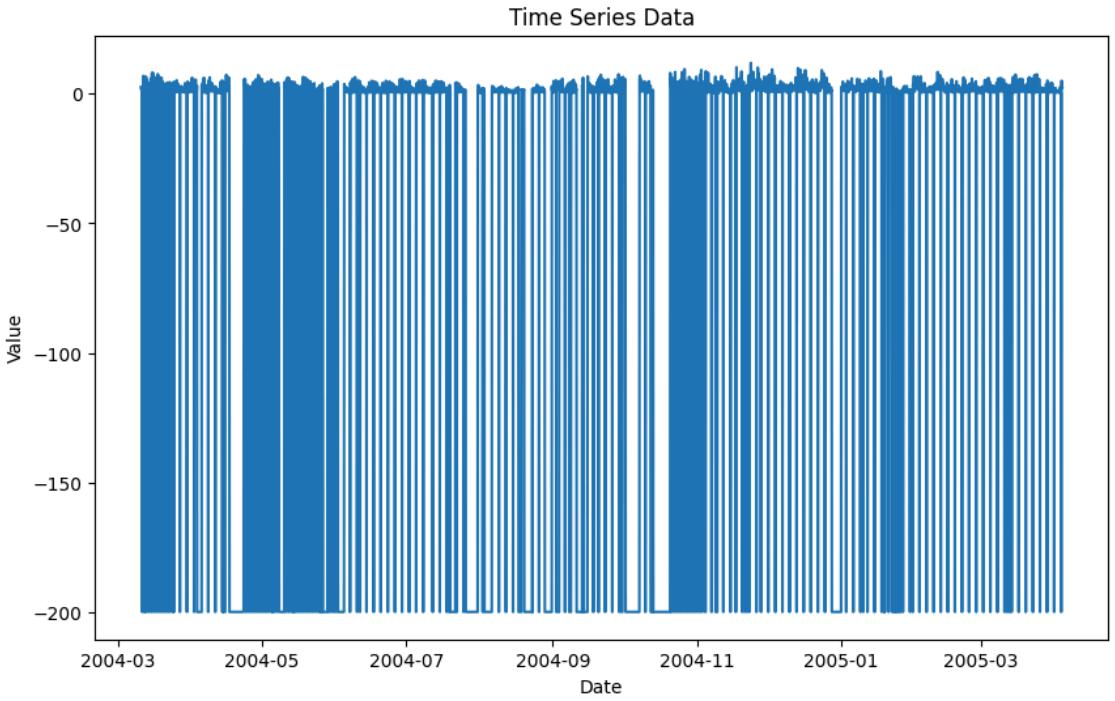
datetime	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	\
2004-03-10 18:00:00	166	1056	113	1692	
2004-03-10 19:00:00	103	1174	92	1559	
2004-03-10 20:00:00	131	1140	114	1555	
2004-03-10 21:00:00	172	1092	122	1584	
2004-03-10 22:00:00	131	1205	116	1490	

datetime	PT08.S5(O3)	T	RH	AH
2004-03-10 18:00:00	1268	13.6	48.9	0.7578
2004-03-10 19:00:00	972	13.3	47.7	0.7255
2004-03-10 20:00:00	1074	11.9	54.0	0.7502
2004-03-10 21:00:00	1203	11.0	60.0	0.7867
2004-03-10 22:00:00	1110	11.2	59.6	0.7888

Step 2)

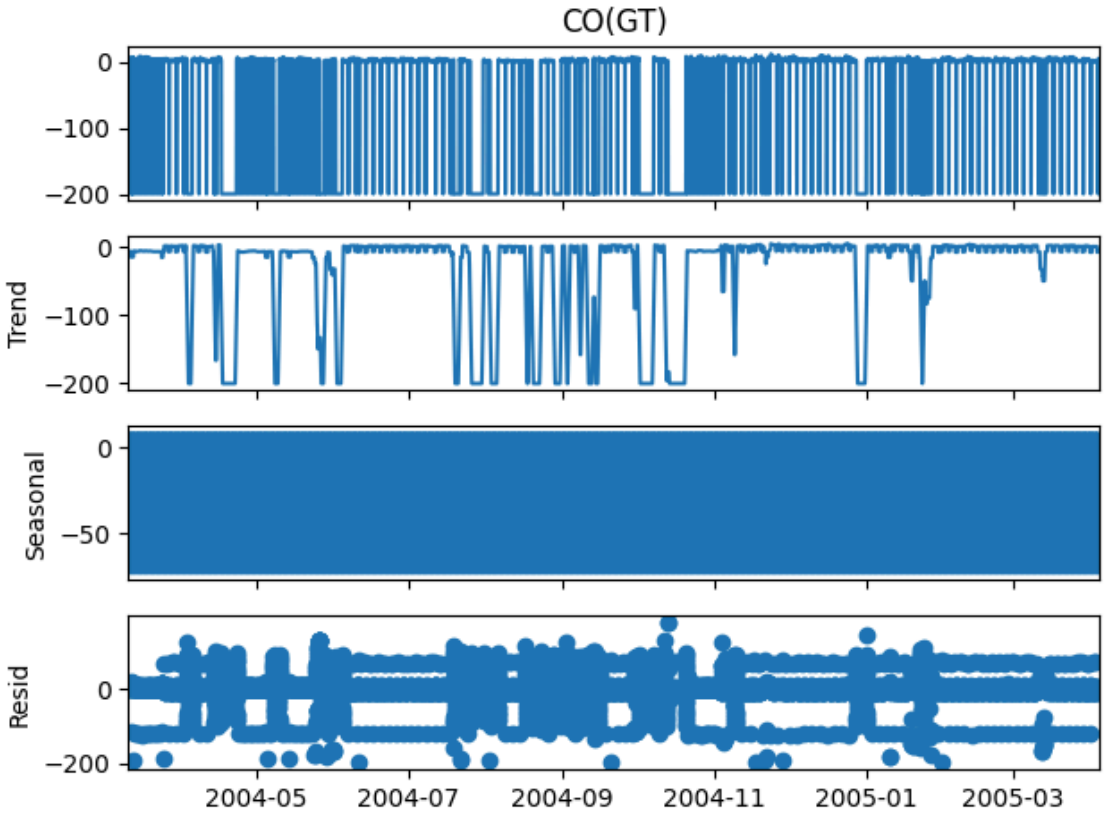
```
[7] series = data['CO(GT)']  # Adjust column names as needed

    # Visualize the data
    plt.figure(figsize=(10, 6))
    plt.plot(series)
    plt.title('Time Series Data')
    plt.xlabel('Date')
    plt.ylabel('Value')
    plt.show()
```



Step 3)

```
[8] # Decompose the data
    decomposition = seasonal_decompose(series, model='additive')
    fig = decomposition.plot()
    plt.show()
```




```
[9] # Fit ARIMA model
    model = ARIMA(series, order=(3, 1, 0))  # Adjust the order as needed
    model_fit = model.fit()

    # Summary of the model
    print(model_fit.summary())
```

```
                               SARIMAX Results
==========================================================================================
Dep. Variable:                       meantemp   No. Observations:                  114
Model:                 ARIMA(3, 1, 0)   Log Likelihood                -218.746
Date:                Thu, 03 Oct 2024   AIC                             445.493
Time:                        12:12:35   BIC                             456.402
Sample:                    01-01-2017   HQIC                            449.920
                         - 04-24-2017
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.1227      0.096     -1.278      0.201      -0.311       0.065
ar.L2          0.0022      0.105      0.021      0.983      -0.204       0.208
ar.L3          0.0249      0.099      0.251      0.802      -0.169       0.219
sigma2         2.8111      0.363      7.741      0.000       2.099       3.523
===================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):                 2.93
Prob(Q):                              0.97   Prob(JB):                         0.23
Heteroskedasticity (H):               0.63   Skew:                            -0.35
Prob(H) (two-sided):                  0.16   Kurtosis:                         3.38
===================================================================================
```

Step 4)

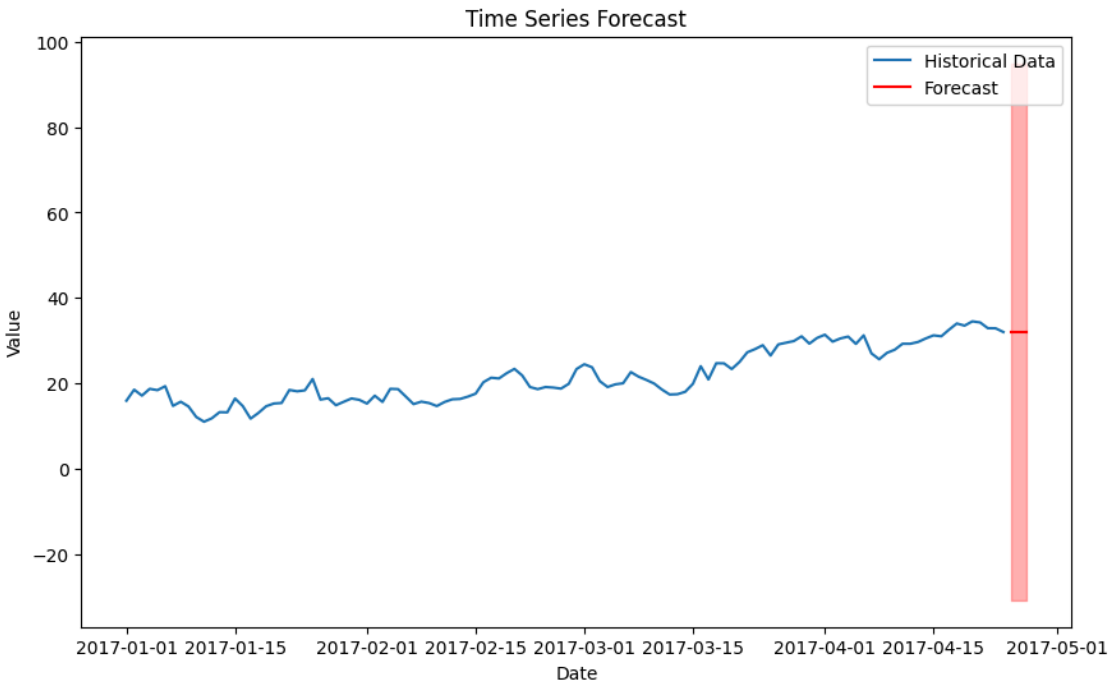
```
# Forecast future values
forecast_steps = 3  # Number of periods to forecast
forecast, stderr, conf_int = model_fit.forecast(steps=forecast_steps)
```

Step 5)

```
# Create a DataFrame for the forecast
forecast_index = pd.date_range(start=series.index[-1] + pd.Timedelta(days=1), periods=forecast_steps, freq='D')
forecast_series = pd.Series(forecast, index=forecast_index)
```

Step 6)

```
# Plot the results
plt.figure(figsize=(10, 6))
plt.plot(series, label='Historical Data')
plt.plot(forecast_series, color='red', label='Forecast')
plt.fill_between(forecast_series.index, forecast_series - 1.96 * stderr, forecast_series + 1.96 * stderr, color='red', al
plt.title('Time Series Forecast')
plt.xlabel('Date')
plt.ylabel('Value')
plt.legend()
plt.show()
```



Date: 17/ 10/ 2024 **Signature of faculty in-charge**

Post Lab Descriptive Questions:

1. What are the key components of a time series, and how do they affect the analysis?

A time series is typically composed of four key components:

- **Trend**: This represents the long-term progression of the series, showing an overall increase or decrease over time. Trends can be linear or non-linear.
- **Seasonality**: These are regular patterns or fluctuations that occur at specific intervals, such as daily, monthly, or yearly. Seasonality reflects periodic influences on the data.
- **Cyclical**: These are long-term fluctuations that occur at irregular intervals, often tied to economic or business cycles, and are not strictly periodic like seasonality.
- **Residual (or Irregular)**: This component captures random noise or irregularities that cannot be explained by the trend, seasonality, or cyclical patterns.

Impact on Analysis: Understanding these components helps in modeling and forecasting. For instance, separating these components allows analysts to identify patterns and make predictions more accurately by focusing on relevant factors without the interference of noise.

2. What is the purpose of decomposing a time series into trend, seasonal, and residual components?

Decomposing a time series into its components (trend, seasonal, and residual) serves several purposes:

Simplification: It simplifies the analysis by isolating patterns, making it easier to understand underlying behaviors.

Forecasting: By modeling each component separately, analysts can improve forecasting accuracy. For example, forecasting seasonal patterns can enhance overall predictions.

Understanding Relationships: It helps in understanding how different components interact. For example, one might analyze how trends are influenced by seasonal patterns.

Anomaly Detection: By examining residuals, analysts can identify outliers or anomalies in the data that may require further investigation.

3. Explain how the ARIMA model works and what the terms (p, d, q) represent.

ARIMA (AutoRegressive Integrated Moving Average) is a popular time series forecasting model that combines autoregressive and moving average components with differencing to make the data stationary.

- **p (AutoRegressive part)**: This parameter indicates the number of lagged observations (previous time points) included in the model. It captures the relationship between an observation and a number of lagged observations.
- **d (Differencing)**: This parameter denotes the number of times the data needs to be differenced to achieve stationarity (i.e., the mean and variance remain

constant over time). Differencing is a technique used to remove trends or seasonality from the data.

- **q (Moving Average part)**: This parameter specifies the number of lagged forecast errors in the prediction equation. It captures the relationship between an observation and a residual error from a moving average model.

In summary, ARIMA models leverage the relationships between past values and past errors to generate forecasts, making them versatile for various types of time series data.